

AI Reference

AI Tool(s) Used

OpenAI ChatGPT / Codex was used as an implementation, debugging, documentation, and deployment assistant for this Cocos Creator 2.4.8 Web Mario assignment.

Project Context

The assignment requires a Mario-style game made with Cocos Creator, including start menu, level select, game scene, game over / restart, physics, player controls, enemies, question blocks, animations, sound effects, UI, appearance, Git commits, README, Firebase deployment, and AI_reference.pdf.

Scope of AI Usage

assets/Scripts/WebMarioGame.js lines 12-138: Cocos component setup, game states, scene nodes, event listeners, and responsive page-fit CSS.

assets/Scripts/WebMarioGame.js lines 141-598: Firebase Realtime Database leaderboard, score submission, top 10 loading, and graceful error handling.

assets/Scripts/WebMarioGame.js lines 600-805: provided asset loading for images, atlases, bitmap fonts, audio, player sprites, enemies, effects, tiles, and UI images.

assets/Scripts/WebMarioGame.js lines 806-999: start menu, level select UI, menu buttons, editable prefab support, and text styling.

assets/Scripts/WebMarioGame.js lines 1001-1214: game initialization, player reset, enemy setup, HUD setup, and Cocos sprite visibility switching.

assets/Scripts/WebMarioGame.js lines 1215-1390: two level maps with ground, walls, pipes, clouds, brick platforms, question blocks, enemies, decorations, and flags.

assets/Scripts/WebMarioGame.js lines 1391-1690: keyboard controls, gravity, collision, camera follow, enemy movement, stomp-only enemy defeat, question block interaction, mushrooms, and player damage.

assets/Scripts/WebMarioGame.js lines 1691-2107: HUD, result overlay, sprite synchronization, tile rendering, decorations, power-up rendering, effects, and animation frame selection.

assets/Scripts/WebMarioGame.js lines 2108-2150: menu and level-select button hitbox alignment using current sprite position and size.

build/web-desktop/index.html, build/web-desktop/leaderboard-ui.js, firebase.json: Firebase Hosting deployment, no-cache headers, standalone leaderboard access, and viewport fit fixes.

README.md and AI_reference.pdf: documentation, completed scoring items, Git notes, deployment link, submission notes, and AI disclosure.

Representative Prompts and Responses

Prompt: Detailed reference to the PDF requirements and complete the assignment.

Response/result: extracted assignment checklist and implemented Cocos Mario game requirements.

Prompt: Use Cocos Creator 2.4.8, not Creator3D. Response/result: changed implementation to Cocos Creator 2.4.8 2D global cc APIs and cc.Class.

Prompt: Use the provided assets, do not draw everything manually. Response/result: replaced placeholders with provided menu art, tiles, Mario sprites, enemy atlases, items, flag, HUD icons, and audio.

Prompt: The level looks too simple; add more walls and parkour elements. Response/result: expanded maps with pipes, brick walls, cloud platforms, gaps, stairs, question blocks, decorations, enemies, and flags.

Prompt: Big Mario should become small when touching an enemy instead of dying immediately. Response/result: modified damage flow so big Mario powers down first, with invincibility and

sound effect.

Prompt: Add Firebase Realtime Database leaderboard. Response/result: added name input, score submission, createdAt timestamp, top 10 display, and console-only failure handling.

Prompt: Deployed leaderboard does not show scores. Response/result: diagnosed Firebase index error, changed loading to read leaderboard.json and sort scores on the client.

Prompt: Deployed button position seems lower than the visible button. Response/result: changed click detection to use actual sprite node position and content size.

Refinement and Explanation

AI suggestions were reviewed and revised through repeated local preview and Firebase deployment tests. Final fixes included asset cropping, mushroom movement, enemy movement, turtle behavior, big Mario damage, leaderboard loading, cache control, and button hitbox alignment.

The game uses rectangle-based platform physics: objects have position, size, velocity, and collision state. Gravity changes vertical velocity. Objects are moved and then corrected against solid blocks. Stomping an enemy from above defeats it; side collision hurts the player.

The Firebase leaderboard uses the Realtime Database REST API. Scores are saved as name, score, and createdAt. The game reads leaderboard.json, sorts scores descending on the client, and displays the top 10. This avoids requiring a Firebase .indexOn rule.

Statement

AI tools were used as described in this report. The final project was reviewed, modified, tested, and integrated by the student in the local Cocos Creator project and Firebase deployment.